

Soil Classifier — Training Plan

EfficientNet-B0 fine-tuning · 4 soil types · pi515 AI Challenge

WHAT ARE WE BUILDING

A model that looks at a photo of soil and identifies what type it is — Alluvial, Black, Clay, or Red. The model will eventually run inside a phone app so a farmer can point their camera at the ground and get an instant result.

WHY THESE 4 TYPES?

These are the four dominant agricultural soil types in India, where our training dataset was collected. Each type has distinct visual characteristics — color, texture, and moisture appearance — that a model can learn to tell apart from a photo.

THE APPROACH — FINE-TUNING

Instead of training a model from scratch (which would need millions of images and days of compute), we use **fine-tuning**. We start with EfficientNet-B0, a model that was already trained on 1.2 million photos and already "knows" how to recognize shapes, textures, and colors. We then swap out only its final decision layer and retrain just that part on our soil images.

ANALOGY

Think of it like hiring an expert photographer who already knows how cameras and lighting work. You just teach them the specific subject — soil — rather than starting from zero.

The model has two parts:

FROZEN — STAYS EXACTLY AS DOWNLOADED, NO CHANGES

Backbone (MBConv blocks) — detects edges, textures, colors

Global Average Pool — compresses image into 1,280 numbers

TRAINABLE — ONLY THIS PART GETS UPDATED

Classifier head — Linear(1280 → 4 soil types) ← we train this

Only ~5,000 of the model's 4 million parameters are actually updated during training. This makes it fast, light, and unlikely to overfit on a small dataset.

DATASET

FIELD	DETAIL
Name	Soil Image Dataset (Kaggle)
Total images	1,553 ground-level soil photos
Classes	Alluvial, Black, Clay, Red
Train/Test split	Already split into Train and Test folders
Validation	20% of the Train folder held back automatically at runtime
Location on disk	<code>Desktop/soil_data/Dataset/</code>

WHAT IS A TRAIN/VAL/TEST SPLIT?

Train — the images the model learns from.

Validation — images checked after each epoch to see if the model is improving (not used for learning).

Test — images the model has never seen, used only at the very end to get a final honest accuracy score.

WHERE WE TRAIN

FIELD	DETAIL
Format	Jupyter Notebook — run cell by cell, see results inline
Machine	Your Mac (local)
GPU	MPS (Apple Silicon Metal GPU) — auto-detected, no setup needed
Framework	PyTorch + torchvision

Est. time	5–8 minutes for 10 epochs
Output	<code>soil_model.pt</code> — best model checkpoint saved automatically

WHY A JUPYTER NOTEBOOK?

A notebook lets you run the code one step at a time, see sample images before training, watch accuracy improve after each epoch, and plot results — all without leaving the browser. Much easier to follow than running a script in the terminal.

TRAINING SETTINGS

These are the "knobs" that control how training runs:

SETTING	VALUE	PLAIN ENGLISH
Epochs	10	How many times the model sees the full dataset. 10 is enough when only the head is training.
Batch size	32	How many images are fed in at once. 32 fits comfortably in Mac memory.
Learning rate	0.001	How big each learning step is. Higher is fine here since the backbone is frozen.
Optimizer	Adam	The algorithm that updates weights. Adam is the reliable default.
Loss function	CrossEntropyLoss	How the model measures how wrong it was. Standard for multi-class problems.
Image size	224 x 224 px	EfficientNet-B0's expected input size. All

photos are resized to this.

DATA AUGMENTATION

To help the model generalize (work well on photos it has never seen), we apply random transformations to training images on the fly. The original images are never changed on disk.

TRANSFORMATION	WHAT IT DOES
RandomResizedCrop	Randomly zooms in and crops a 224px square — simulates different distances from the ground
RandomHorizontalFlip	Mirrors the image left/right — soil looks the same upside down
RandomVerticalFlip	Mirrors top/bottom — same reasoning
ColorJitter	Slightly shifts brightness and color — handles different lighting and shadows
Normalize	Scales pixel values to match what the backbone expects from ImageNet training

STEP-BY-STEP PLAN

- 1 Open the notebook**
Launch Jupyter from the terminal and open `train.ipynb` in the `soil_classifier` folder.
- 2 Preview the data**
The first cells load and display sample images from each class so you can visually confirm the dataset loaded correctly.
- 3 Load the model**
EfficientNet-B0 is downloaded automatically from PyTorch (about 20 MB). The backbone is frozen and the head is replaced.
- 4 Train**
Run the training cell. After each of the 10 epochs you'll see the training loss and validation accuracy printed live.
- 5 Evaluate on test set**

The final cell runs the saved best model against the held-out test images and prints accuracy per class.

6 Export for mobile

Convert `soil_model.pt` to CoreML (iPhone) or TFLite (Android)
— covered in a separate export notebook.

EXPECTED RESULTS

METRIC	EXPECTED RANGE
Validation accuracy	80 – 90%
Test accuracy	75 – 88%
Training time	5 – 8 minutes on Mac MPS
Model file size	~16 MB

ACTUAL RESULTS — TRAINED 2026-04-10

MODEL EXCEEDED EXPECTATIONS

Final test accuracy came in at **92.65%** — above the 75–88% expected range. Trained in 2 minutes 30 seconds on Mac MPS (Apple Silicon GPU).

METRIC	EXPECTED	ACTUAL
Best val accuracy	80 – 90%	92.62%
Test accuracy	75 – 88%	92.65%
Training time	5 – 8 min	2 min 30 sec
Device	Mac MPS	MPS (Apple Silicon)

PER-CLASS RESULTS

SOIL TYPE	PRECISION	RECALL	F1 SCORE
Alluvial Soil	83.3%	83.3%	83.3%
Black Soil	95.6%	94.0%	94.8%
Clay Soil	88.9%	86.2%	87.5%

Red Soil

96.3%

100.0%

98.1%

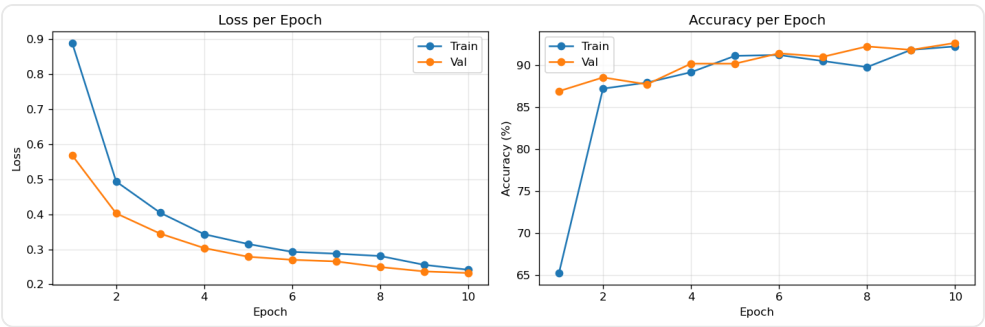
NOTE ON ALLUVIAL SOIL

Alluvial soil had the lowest scores (83.3%) likely because it visually overlaps with Clay soil — both tend to be light-colored and fine-grained. This is a known challenge with this dataset and can be improved in future iterations with more targeted data collection.

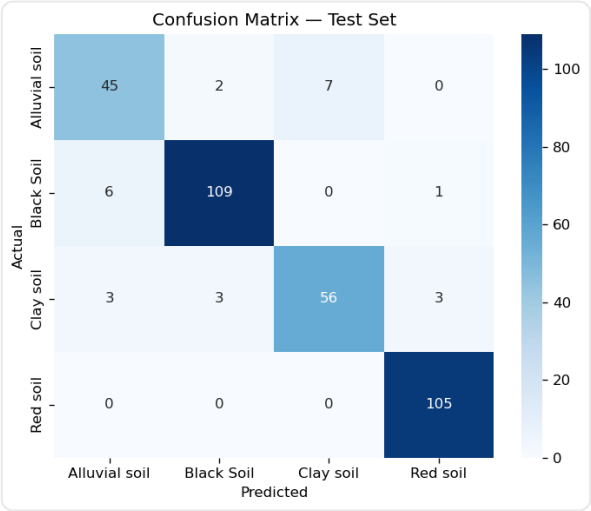
EPOCH-BY-EPOCH TRAINING HISTORY

EPOCH	TRAIN LOSS	TRAIN ACC	VAL LOSS	VAL ACC
1	0.8889	65.3%	0.5683	86.9%
2	0.4934	87.2%	0.4025	88.5%
3	0.4043	87.9%	0.3447	87.7%
4	0.3430	89.1%	0.3035	90.2%
5	0.3150	91.1%	0.2787	90.2%
6	0.2926	91.2%	0.2700	91.4%
7	0.2875	90.5%	0.2654	91.0%
8	0.2808	89.8%	0.2493	92.2%
9	0.2556	91.8%	0.2369	91.8%
10 ✓	0.2416	92.2%	0.2324	92.6%

TRAINING CURVES



CONFUSION MATRIX



Each row = actual class · Each column = predicted class · Diagonal = correct predictions